
FATE: 异构 LLM 工作流的未来状态感知调度

Zirui Huang, Yi-Xiang Hu, Feng Wu, Xiang-Yang Li

School of Computer Science and Technology

University of Science and Technology of China, Hefei, China

{hzh090901,yixianghu}@mail.ustc.edu.cn, {wufeng02,xiangyangli}@ustc.edu.cn

摘要

大语言模型 (LLM) 应用日益以异构多阶段工作流执行, 而非彼此孤立的推理调用。在这些工作流有向无环图 (DAG) 中, 调度决策不仅影响当前就绪阶段, 还影响下游阶段继承的执行状态, 包括模型驻留、父节点输出局部性、前缀复用和未来设备可达性。现有服务与 DAG 调度策略主要分别优化即时队列状态、放置成本或复用信号, 可能使有用状态碎片化, 增加端到端延迟。我们提出 FATE, 一种面向异构 LLM 工作流的未来状态感知调度器。FATE 结合 CP-SAT 支持的前沿规划器、感知预测范围的候选评分、有界多设备分片执行及以状态为条件的成本估计。FATE 不求解覆盖整个 DAG 的单体问题, 而是反复对当前就绪前沿规划, 并按即时成本及分配所诱导的下游状态共同评分。在由 WfCommons 衍生的工作流 DAG 基准和受控前缀复用基准中, FATE 优于实用启发式、经典 DAG 调度, 以及近期工作流服务策略的代理改编。在工作流 DAG 基准上, 其归一化 makespan 和归一化 P95 延迟分别为 0.675 与 0.677, 相对 RoundRobin 降低 32.5% 与 32.3%, 相对最强的非 FATE 基线降低 8.9% 与 8.8%。机制分析和消融说明, 这些收益来自联合保留多个未来执行状态维度, 而非单独的前缀复用。结果表明, 保留未来状态是异构 LLM 工作流服务中有用的调度目标。

1 引言

大语言模型 (LLM) 服务近期进展显著提高了单次推理调用的效率。分页 KV 缓存管理、结构化语言程序执行、预填充/解码调度和架构拆分系统, 改善了吞吐量、延迟及内存效率 [1–4]。当工作负载可以由单次模型调用或弱耦合调用序列良好近似时, 这些技术非常有效。

现代 LLM 应用日益打破这一抽象。单个用户查询可以触发包含检索、分解、路由、工具使用、代码执行、验证、聚合及后处理的依赖工作流。这一模式在智能体助手和软件工程助手中尤为明显, 高层请求可被分解为专门的规划、实现、测试、审查与修复阶段。这些应用自然表示为具有异构阶段、异构模型和显式跨阶段依赖的有向无环图 (DAG)。近期系统开始直接暴露这种结构, 将智能体应用视为工作流服务问题, 而非独立请求 [5–7]。

这一变化从根本上改变了调度问题。传统服务中, 调度器主要响应队列状态: 哪些请求就绪、哪个设备空闲, 以及如何安排批处理。而在异构工作流服务中, 调度器还塑造下游阶段继承的执行状态。将阶段放到某个设备而非另一个设备, 可能保留模型驻留、避免未来传输、改善父子共置, 或为下游阶段保留可复用的前缀相关状态 [8, 9]。反过来, 激进地均衡当前负载, 可能破坏对端到端工作流延迟更重要的局部性。

因此, 我们认为异构 LLM 工作流调度除了即时执行效率, 还应优化未来状态保留。我们将所得调度器称为 FATE, 名称取自未来状态感知 (Future-stATE-aware) 调度。图 1 给出概览。给定异构工作流 DAG 和当前执行状态, FATE 反复为当前就绪前沿选择分配方案, 并按每个候选会产生的未来执行状态评分: 哪个模型继续驻留、哪些父节点输出留在本地、哪些前缀与下游阶段保持对齐, 以及哪些后续阶段仍可到达而不发生可避免的传输。有限的下游预测范围估计这些后果, 但规划器仅提交当前前沿, 随后推进并更新状态。与 HEFT [10]

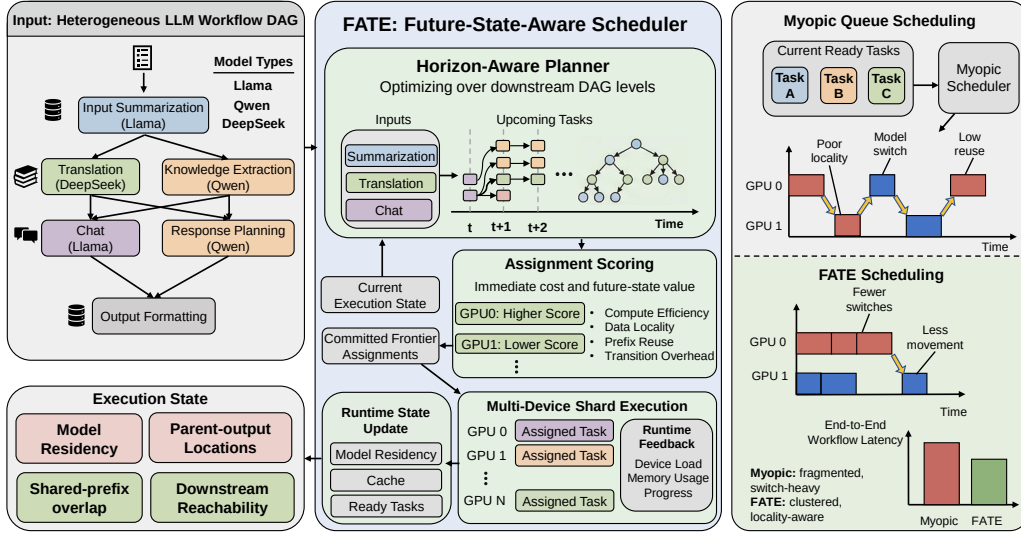


图 1: FATE 概览。给定异构 workflow DAG 与当前执行状态, FATE 进行感知预测范围的规划, 为可行阶段—设备分配评分, 执行有界多设备分片放置, 并随着执行展开更新执行状态。与短视、仅考虑队列的调度不同, FATE 同时优化即时成本与未来状态质量。

等纯贪心或基于排序的 DAG 调度器不同, FATE 明确将状态依赖的下游成本视为调度目标的一部分 [5, 6, 8]。

经验上, 我们发现两个稳健结果。首先, 执行状态感知调度在由 WfCommons 衍生的 workflow DAG 基准中改善端到端性能 [11]。其次, 这些收益不能仅用前缀复用解释: 受控前缀复用测试中, 复用感知基线缩小了差距, 但未消除差距。两者共同说明, 在所研究的工作流服务区间, 未来状态保留是有用的调度目标。

主要贡献如下:

- 我们识别出执行状态是异构 LLM 工作流中 useful 且常被优化不足的目标。传统服务的调度主要响应队列状态, 工作流执行却依赖不断变化的模型驻留、跨阶段局部性、可复用前缀相关状态和受约束的未来可达性。
- 我们形式化异构 workflow DAG 的执行状态感知放置与评分, 捕获设备约束下的阶段放置、有界多设备执行、切换成本、传输成本及依赖状态的执行收益。
- 我们用 CP-SAT 支持的调度器 [12] 实现这一形式化, 结合感知预测范围的规划和状态条件评分, 并表明核心权衡并非单纯获得更好的均衡, 而是均衡与保留未来状态之间的权衡。
- 我们引入统一的异构多 LLM 工作流调度框架, 显式暴露这些状态变量, 使未来状态感知策略、实用启发式、经典 DAG 调度器, 以及近期服务策略的代理改编, 能够在同一工作流运行时中评估。

2 问题形式化

我们考虑一个在线系统, 服务动态变化的工作流实例集合。令 $\mathcal{W}_t$ 表示时刻 t 的活跃工作流集合。为清晰起见, 我们对通用工作流实例定义阶段级及工作流级量, 在含义明确时省略工作流索引。因此, 运行时就绪集合、前沿集合和可行动作集合, 均理解为 $\mathcal{W}_t$ 中当前活跃实例的各工作流对应集合的并集。每个工作流实例表示为 DAG $G = (V, E)$, 节点 $v \in V$ 表示一个工作流阶段, 有向边 $(u, v) \in E$ 表示先后依赖, 即阶段 v 只有收到阶段 u 的输出后才能开始。

对每个阶段 v , 令

$$(m(v), A(v), R(v), c_v, \phi(v), \text{Pa}(v), \text{Ch}(v))$$

表示其属性，其中：(i) $m(v)$ 是 v 使用的模型类型；(ii) $A(v) \subseteq \mathcal{D}$ 是有资格执行它的设备集合；(iii) $R(v) \in \mathbb{Z}_{\geq 1}$ 是有界多设备分片执行的最大并行度；(iv) $c_v : A(v) \rightarrow \mathbb{R}_+$ 是各合格设备上的基础运行时间配置，满足

$$c_{\text{base}}(v, d) = c_v(d), \quad d \in A(v);$$

(v) $\phi(v)$ 表示阶段局部特征，包括提示元数据、共享前缀组信息、缓存相关标志及运行时阶段注释；(vi) $\text{Pa}(v)$ 和 $\text{Ch}(v)$ 表示 v 的父阶段与子阶段。

时刻 t 的执行状态记为

$$s_t = (\rho_t, \kappa_t, \ell_t, \tau_t),$$

其中 ρ_t 编码每个设备上的模型驻留， κ_t 编码每个设备上可复用的前缀相关元数据， ℓ_t 记录已完成父节点输出的设备位置， τ_t 记录设备可用时间。因此， s_t 总结影响未来切换、传输、局部性与复用的执行状态。

令 $\mathcal{C}_t \subseteq V$ 表示时刻 t 已完成阶段的集合。就绪集合为

$$\mathcal{R}(s_t) = \{v \in V \setminus \mathcal{C}_t : \text{Pa}(v) \subseteq \mathcal{C}_t\}.$$

可行候选集合为

$$\mathcal{F}(s_t) = \{(v, d) : v \in \mathcal{R}(s_t), d \in A(v), (v, d) \text{ satisfies current resource feasibility}\}.$$

我们寻求策略 π ，联合决定阶段放置、有界分片执行、执行次序和规划决策，以最小化端到端 workflow 完成时间：

$$\min_{\pi} \mathbb{E}_{W \sim \mathcal{P}} [L(W; \pi)],$$

其中 $L(W; \pi)$ 表示策略 π 下 workflow W 的完成时间。

关键区别是执行成本依赖状态，而非固定。因此，将状态 s_t 下阶段 v 放置到设备 d 的有效成本写为

$$\hat{c}(v, d, s_t) = g(c_v(d), m(v), \phi(v), \text{Pa}(v), \text{Ch}(v), s_t),$$

其中 $g(\cdot)$ 捕获当前执行状态如何通过模型驻留、父节点输出局部性、前缀相关复用、传输成本和有限分片执行效果，修改名义阶段—设备运行时间。这一状态依赖成本的具体分解取决于方法；FATE 用第 3 节的规划分数、调度分数和校正项实现它。因此，问题不仅异构，还依赖状态：时刻 t 的决策改变未来执行状态，进而改变下游切换成本、传输成本、局部性和复用机会。

3 方法

FATE 基于一个简单原则：调度决策除了优化即时执行效率，还应优化其诱导的未来执行状态质量。这一视角统一了常被作为独立启发式处理的多种效果，包括模型局部性、父子共置、前缀相关复用和传输最小化。

3.1 概览

FATE 包含三个组件：(i) CP-SAT 支持的前沿规划器 [12]，候选评分纳入有限下游预测范围；(ii) 增量式提交并推进的规划流程，在构建调度期间逐层向前移动预测范围；(iii) 以状态为条件的成本与评分模型，根据当前系统状态调整执行估计。

每个规划步骤中，FATE 利用有限下游层级视图产生的、感知预测范围的候选分数，评估当前就绪前沿。随后调用 CP-SAT 规划器，在当前状态下产生阶段放置及有界多设备执行的候选决策。选定放置被具体化为可执行的阶段—设备分配，运行时执行器按构建的逐设备执行次序发出依赖已就绪任务。FATE 不精确求解完整的状态依赖调度问题，而是通过实用分解实现未来状态保留：前沿规划、基于就绪性的执行，以及以状态为条件的成本校正。

3.2 未来状态感知分解

FATE 的主要设计原则是：评价调度动作时，既考虑其即时执行成本，也考虑它诱导的未来执行状态质量。概念上，这一权衡可写为

$$Q(s_t, a_t) = -C_{\text{imm}}(s_t, a_t) + \gamma V_{\text{future}}(s_{t+1}),$$

其中 $C_{\text{imm}}(s_t, a_t)$ 表示即时执行成本， $V_{\text{future}}(s_{t+1})$ 总结所得执行状态的下游价值。

FATE 不学习，也不直接优化这一价值函数，而是用该分解组织实用近似：逐阶段规划器的候选分数包有限层级预测范围内的下游尾部估计，同时配合调度分数和成本校正项，考虑模型驻留、与依赖对齐的放置、前缀相关复用及有界分片并行。

3.3 感知预测范围的前沿规划

为明确分析未来状态权衡，FATE 使用逐阶段 CP-SAT 规划器。规划按下游 DAG 层级索引，但求解器本身只在当前就绪前沿调用，并不同时覆盖全部未来层级。

我们在候选评分步骤使用有界预测范围 H 。实现中， $H = 1$ 为当前就绪前沿评分，较大值则加入最多 $H - 1$ 个下游 DAG 层级的折扣估计。FATE 不把未来阶段展开为显式 CP-SAT 变量，而是将其估计需求与复用效果折叠到附在当前前沿候选上的、感知预测范围的分数中。

在规划步骤 j ，令 $\mathcal{R}_j$ 表示当前就绪前沿。对每个就绪阶段 $v \in \mathcal{R}_j$ 、分片槽位 $k \in \{0, \dots, R(v) - 1\}$ 和合格设备 $d \in A(v)$ ，规划器引入二元分配变量 $y_{v,k,d}$ ，表示阶段 v 的分片槽位 k 是否放到设备 d 。前沿规划器求解

$$\max_y \sum_{v \in \mathcal{R}_j} \sum_{k=0}^{R(v)-1} \sum_{d \in A(v)} y_{v,k,d} \Psi(v, k, d \mid s_t, H),$$

其中 $\Psi(v, k, d \mid s_t, H)$ 为状态感知放置分数，结合即时执行质量与有限预测范围内的下游尾部估计。

规划器施加设备资格、设备容量和有界分片约束。具体而言，每个规划步骤中，每个设备最多接收一个前沿分配；分片槽位 0 作为阶段的主分配，只有较低索引槽位已启用，才能启用更高索引槽位。这些约束使 FATE 能建模有界多设备执行，同时防止不受约束的复制或推测竞速。有界分片执行把阶段的查询批次划分到选定设备，并不将单个查询拆分到多个设备。

在当前前沿求解后，FATE 将选定分配具体化为可执行的阶段—设备放置。已分配阶段完成时，更新执行状态；当前前沿耗尽后，规划器重新计算下一个就绪前沿并重复。

因此，规划器采用滚动、逐阶段方式：未来层级通过感知预测范围的分数影响当前选择，但运行时不会对整个 DAG 求解一个单体优化问题。完整决策变量、约束和实现细节见附录 A。

3.4 状态感知评分

规划器决定分片槽位放置，并将其具体化为可执行的阶段—设备分配。为比较当前执行状态下的可行分配，FATE 使用未来状态感知分数。

对候选阶段—设备分配 (v, d) ，FATE 计算

$$\begin{aligned} S(v, d \mid s_t) = & -\lambda_q C_{\text{wait}}(v, d, s_t) - \lambda_s C_{\text{switch}}(v, d, s_t) - \lambda_{\text{tr}} C_{\text{transfer}}(v, d, s_t) \\ & + \lambda_c B_{\text{colo}}(v, d, s_t) + \lambda_p B_{\text{prefix}}(v, d, s_t) + \lambda_r B_{\text{parallel}}(v, d, s_t). \end{aligned}$$

各项捕获等待成本、模型切换成本、跨设备传输成本、父子共置收益、前缀相关复用收益，以及有界多设备分片执行收益。

FATE 在调度构建期间用这一分数排序可行阶段—设备候选：

$$(v^*, d^*) = \arg \max_{(v,d) \in \mathcal{F}(s_t)} S(v, d \mid s_t),$$

其中 $\mathcal{F}(s_t)$ 为第 2 节定义的可行候选集合。实现中，选定分配具体化为逐设备执行队列；已分配设备可用时，运行时执行器发出依赖已就绪的阶段。各分数项的详细定义见技术附录。

算法 1 FATE: 未来状态感知调度

```
1: 初始化执行状态  $s_0$  和逐设备执行队列
2: 当 活跃工作流实例中仍有未分配阶段 执行
3:   构建当前就绪前沿  $\mathcal{R}_j$ 
4:   计算前沿候选分数  $\Psi(v, k, d \mid s_t, H)$ 
5:   在  $(s_t, \mathcal{R}_j)$  上求解前沿 CP-SAT 规划器
6:   将选定分片槽位放置具体化为逐设备执行队列
7:   推进规划前沿并更新规划状态
8: 结束 当
9: 当 仍有未完成的可执行阶段 执行
10:  已分配设备可用时, 发出队列中依赖已就绪的阶段
11:  阶段完成时更新执行状态  $s_t$ 
12: 结束 当
```

3.5 以状态为条件的成本估计

规划器与基于就绪性的调度流程所用分数项, 来自共享的状态条件成本估计器。给定阶段—设备对 (v, d) 和状态 s_t , 估计器从名义运行时间配置 $c_v(d)$ 出发, 对模型驻留、父节点输出放置、前缀元数据、队列压力及有界分片执行应用校正。这些校正估计实现规划器分数 $\Psi(v, k, d \mid s_t, H)$ 和调度分数 $S(v, d \mid s_t)$ 中的成本与收益项。

这一分离很重要: 规划器用估计器在有限下游前瞻下比较前沿分配, 执行器则使用所得阶段—设备次序, 并受运行时依赖就绪性约束。因此, 估计器不是第三个调度目标, 而是同时向规划与调度构建提供状态依赖成本的共同测量层。

算法 1 总结整体调度流程。完整决策变量、约束、调度分数项定义和流程细节见附录 A。

4 实验

本节给出全面实验评估, 回答以下问题。首先, FATE 是否相对强实用启发式、经典 DAG 调度及近期工作流服务策略的代理改编, 改善端到端工作流性能? 其次, 不同结构工作组和受控前缀复用设置下, 收益是否一致? 第三, 哪些机制级变化(例如模型切换、跨设备依赖、前缀/局部性代理指标)能够解释观察到的改善?

4.1 实验设置

基准。 我们在两类基准上评估 FATE。首先, 构建基于 WfCommons/WfInstances 的基准 [11]。目前还没有广泛采用、专门面向异构多 LLM 工作流调度的基准, 因此我们将工作流 DAG 转换为 LLM 阶段执行图, 构建结构化评估基础。转换保留所得阶段组之间的依赖结构, 并给每个阶段附加预定义角色模板、模型分配和运行时代理配置, 用于执行、缓存和跨设备传输。模板在评估前固定, 并在各方法中一致使用。附录 C.1 给出转换流程、阶段角色模板、模型分配规则、运行时代理构建及前缀复用生成细节。其次, 将工作流式 DAG 模板与受控长上下文工作负载配对, 构建受控前缀复用基准, 共享前缀重复比例设为 0、0.25、0.5 和 1.0。

遵循 Halo 的批查询工作流服务设置 [13], 每个基准项目为一个工作流模板, 在一批独立查询实例上评估。每个模板采用两种批大小配置: 16 和 32 个独立查询实例, 以检验更高批工作流并发下的稳健性。

比较方法。 我们在相同运行时框架、硬件设置和工作流模板下, 将 FATE 与四类基线比较。主要工作流服务基线为 **Halo** [13], 采用其原始基于搜索的调度算法实现。此外包括简单派发基线 **RoundRobin**、感知未来复用的缓存导向策略 **KVFlow** [8]、异构性感知放置策略 **Helix** [9], 以及经典 DAG 调度基线 **HEFT** [10]。所有方法均在统一工作流服务评估框架中、相同执行环境下实现。

实现概述。 FATE 在下游 DAG 层级上使用逐阶段 CP-SAT 规划、感知预测范围的前沿提交与推进, 以及等待、切换、传输、共置、前缀复用和有界分片并行的显式状态感知分数项; 并细化模型驻留、切换开销和父节点就绪性的运行时间成本模型。对于深层异构工作

表 1: workflow DAG 基准总体结果。归一化 makespan 和归一化 P95 查询完成延迟逐 workflow 相对 RoundRobin 计算, 并用几何平均聚合。X-Dev Edge、Cache Score、Model Cont. 为逐任务机制代理, 正式定义见附录 C.2。归一化指标及 X-Dev Edge 越低越好; Cache Score 和 Model Cont. 越高越好。

方法	端到端		机制		
	归一化 MS↓	归一化 P95↓	跨设备边 ↓	缓存分数 ↑	模型延续 ↑
FATE (本文)	0.675	0.677	1.191	0.129	0.573
KVFlow	0.748	0.750	1.417	0.124	0.492
Helix	0.741	0.742	1.235	0.107	0.460
Halo	0.902	0.907	1.242	0.081	0.374
HEFT	0.791	0.791	1.433	0.073	0.449
RoundRobin	1.0	1.0	1.557	0.075	0.332

表 2: 受控前缀复用研究。数值为相对重复比例 0 时 Halo 归一化的几何平均 makespan。越低越好。

方法	0	0.25	0.5	1.0
Halo	1.000	0.991	0.984	0.981
KVFlow	0.702	0.695	0.693	0.690
FATE	0.596	0.594	0.593	0.587

表 3: 转换后 workflow DAG 上的消融研究。退化幅度相对完整 FATE。

变体	归一化 MS↓	退化
完整 FATE	0.675	—
无未来规划	0.748	+10.82%
无局部性项	0.706	+4.63%
无同模型奖励	0.690	+2.20%
无前缀项	0.685	+1.41%
无分片并行	0.687	+1.82%

流, 实现进一步使用按模型专门化的放置偏好; 对于缓存占主导的同模型场景, 在有益时启用有界多设备分片执行。

指标。 主要端到端指标为归一化 makespan 和归一化 P95 查询完成延迟, 均逐 workflow 相对 RoundRobin 计算, 并跨实例用几何平均聚合。机制诊断报告三个逐任务代理指标: X-Dev Edge、Cache Score 和 Model Cont.。X-Dev Edge 是跨设备父节点边数除以 workflow 任务总数; Cache Score 是估计前缀缓存命中数除以任务总数; Model Cont. 是同模型延续次数除以任务总数。它们是诊断代理, 非端到端目标: X-Dev Edge 近似传输压力, Cache Score 近似前缀/缓存亲和性, Model Cont. 近似模型驻留连续性。正式指标定义见附录 C.2。

4.2 主要结果

表 1 给出 workflow DAG 基准主要结果。FATE 在归一化 makespan 与归一化 P95 延迟上均取得最佳端到端性能。其归一化 makespan 为 0.675, 相比 Helix 的 0.741、KVFlow 的 0.748、HEFT 的 0.791、Halo 的 0.902 和 RoundRobin 的 1.000。尾部性能具有相同趋势: FATE 的归一化 P95 延迟为 0.677, 优于 Helix 的 0.742 和 KVFlow 的 0.750。

机制级诊断帮助解释收益。FATE 达到最低 X-Dev Edge 分数, 表明跨设备父依赖更少; 同时达到最高 Cache Score 与 Model Cont., 表明前缀/缓存亲和性及模型驻留连续性更强。趋势符合未来状态感知调度的预期行为: 尽可能保留有用执行状态, 避免可避免的跨设备碎片化, 并保持下游复用机会, 而非仅优化即时步骤。

图 2 展示逐 workflow 相对 RoundRobin 的归一化 makespan 分布。ECDF 将 FATE 与 KVFlow、Helix、Halo 和 RoundRobin 比较。分布视角确认, 总体改善并非只由少量有利案例驱动; FATE 使大部分 workflow 流向较低归一化 makespan。

实际规划开销较小。在所评估工作负载下, 每次 CP-SAT 求解均在 0.08s 内完成, 相对端到端 workflow 完成时间很小。这表明收益并未伴随难以承受的调度成本。

4.3 受控前缀复用与消融研究

受控前缀复用。 表 2 评估直接操纵共享前缀重复比例的受控前缀复用基准。与 workflow DAG 基准不同, 本测试主要用于行为分析: 隔离检验单独增加前缀复用是否解释性能差距。

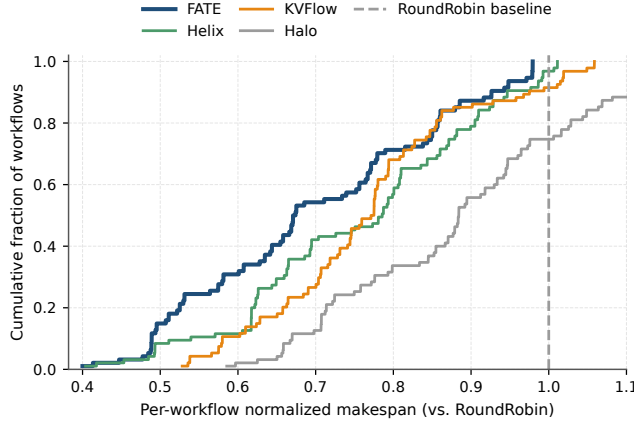


图 2: 工作流 DAG 基准中逐工作流归一化 makespan 的 ECDF。横轴为每个工作流相对 RoundRobin 归一化的 makespan，纵轴为工作流累计比例。曲线越靠左，性能越好。

所有值相对重复比例 0 时的 Halo 归一化，因此列间变化同时反映调度器差异和增加共享前缀可用性的影响。

FATE 在全部重复比例下始终最强，归一化 makespan 保持在 0.587 至 0.596。KVFlow 作为复用感知基线，表现符合预期且具有竞争力，但始终落后于 FATE：重复比例 0 时为 0.596 相对 0.702，重复比例 1.0 时为 0.587 相对 0.690。不同重复比例间变化较小，也说明前缀复用本身并非本测试端到端工作流性能的主导解释。FATE 优势来自共同保留未来模型驻留、依赖局部性和分片机会的调度决策，而非单独缓存复用。

消融。 表 3 评估 FATE 主要组件贡献。无未来规划变体是强内部基线：保留相同状态感知成本项，却移除下游前瞻。由 0.675 退化至 0.748，是最大消融退化，说明收益并非仅来自添加局部性或缓存感知项，而是来自将它们用于未来感知规划。移除局部性项也使性能退化至 0.706，支持父子放置连续性与传输感知调度的重要性。

其他消融表明，前缀感知、同模型偏好和有界分片并行各自提供较小但可测量的收益。移除前缀感知项使归一化 makespan 增至 0.685；移除分片并行增至 0.687；移除同模型奖励增至 0.690。这些结果符合本文更广泛的论点：没有单一机制足以解释全部改善。FATE 有效，是因为它共同保留多个执行状态维度。

4.4 主要认识

实验支持的结论比简单的“求解器击败启发式”更精确。在 WfCommons 衍生工作流 DAG 基准中，FATE 改善端到端性能，使逐工作流 makespan 的 ECDF 移向更低延迟，并在移除未来状态规划时退化最多。受控前缀复用下，它也优于单独复用感知调度。这些结果应理解为工作流状态感知调度在受控、经转换的 DAG 评估中有效的证据，而非全面领先生产系统的声明。机制诊断与消融表明，收益来自共同平衡模型驻留、依赖局部性、传输结构、前缀复用和有界分片执行。

5 相关工作

LLM 服务系统。 近期 LLM 服务系统通过 KV 缓存管理、结构化执行、阶段感知调度和拆分提高效率。代表包括引入 PagedAttention 实现高效 KV 内存管理的 vLLM [1]；用于结构化程序执行与复用的 SGLang [2]；用于分块预填充调度的 Sarathi-Serve [3]；以及采用拆分或 KVCache 中心服务架构的 DistServe 和 Mooncake [4, 14]。这些系统显著提高推理效率，但主要针对单请求或请求级服务，而非异构多阶段工作流 DAG。

工作流感知与智能体服务。 近期工作认识到，许多 LLM 应用以依赖工作流执行，而非孤立调用 [5–7]。Halo 将此类工作负载形式化为查询计划 DAG，在工作流层面优化执行 [13]。KVFlow 研究工作流感知前缀/KV 管理，着重保留未来工作流步骤可能复用的条目 [8]。

Helix 强调异构性感知服务与放置 [9]。我们的工作与这一研究线最接近，但范围不同：不只关注复用或放置，而是研究异构设备、状态依赖切换与传输成本，以及有界多设备分片执行下的执行状态感知放置和调度。

异构 DAG 调度。 HEFT [10] 等经典 DAG 调度是自然参照，因为工作负载是异构设备上执行的结构化 DAG。但传统任务图的执行与通信成本大多静态，LLM 工作流阶段却通过模型驻留、父子局部性及前缀相关复用诱导强烈的状态依赖成本。这使静态、基于排序的调度在许多区间不足，促使调度器明确推理当前决策产生的未来执行状态。

6 局限

本工作有以下局限。首先，尽管在工作流 DAG 和受控前缀复用基准上评估 FATE，当前研究尚未覆盖全部部署条件，如突发到达或更强配置漂移。更广泛地说，目前还没有广泛采用、专门面向异构多 LLM 工作流调度的基准。因此，当前评估从 WfCommons/WfInstances 派生真实 DAG 工作负载，将其转换为折叠后的 LLM 阶段执行图，并附加阶段角色、模型分配及运行时成本代理；受控前缀复用结构则另外引入，用于行为分析。这提供实用、结构化评估基础，却尚非社区标准基准。其次，KVFlow、Helix 和 HEFT 风格比较，是共同工作流服务框架中的策略改编，而非原服务栈的完整系统复现。第三，部分机制诊断是代理指标，并非直接硬件计数器：跨设备父边近似传输压力，而非测量实际传输字节；前缀/局部性代理近似复用行为，而非测量真实 KV 缓存命中率。这些代理仅用于机制分析，主要结论基于归一化 makespan、工作流完成时间等直接端到端指标。

7 结论

本文研究一个在现代 LLM 系统中日益重要的问题：应用以异构多阶段工作流执行时，调度器应优化什么？我们认为，执行状态——不断变化的模型驻留、父子局部性、可复用前缀结构和受约束的下游可达性配置——是异构工作流调度中即时负载与队列状态的重要补充。

我们形式化异构工作流 DAG 的执行状态感知放置与评分，并用 FATE 实现：一个 CP-SAT 支持、结合感知预测范围规划与状态条件评分的调度器。经验上，FATE 在工作流 DAG 和受控前缀复用基准中，优于强实用启发式、经典 DAG 调度，以及近期工作流服务策略的代理改编。收益伴随更少跨设备父边、更高同模型延续及更强前缀/局部性代理，表明收益来自保留有用未来执行状态，而非仅优化当前步骤。

核心认识是，未来状态保留应被视为异构 LLM 工作流服务的重要优化目标。联合推理切换、局部性与复用的调度器，能够优于仅分别优化某一因素的基线。

参考文献

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [2] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 117–134, 2024.
- [4] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

- [5] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. Autelix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- [6] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. Murakkab: Resource-efficient agentic workflow orchestration in cloud platforms. *arXiv preprint arXiv:2508.18298*, 2025.
- [7] Marco Laju, Donghyun Son, Saurabh Agarwal, Nitin Kedia, Myungjin Lee, Jayanth Srinivasa, and Aditya Akella. Nalar: An agent serving framework. *arXiv preprint arXiv:2601.05109*, 2026.
- [8] Zaifeng Pan, AJJKUMAR PATEL, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. Kvflow: Efficient prefix caching for accelerating llm-based multi-agent workflows. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [9] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 586–602, 2025.
- [10] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE transactions on parallel and distributed systems*, 13(3):260–274, 2002.
- [11] Tainã Coleman, Henri Casanova, Loïc Pottier, Manav Kaushik, Ewa Deelman, and Rafael Ferreira da Silva. Wfcommons: A framework for enabling scientific workflow research and development. *Future generation computer systems*, 128:16–27, 2022.
- [12] Laurent Perron, Frédéric Didier, and Steven Gay. The cp-sat-lp solver (invited talk). In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, pages 3–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023.
- [13] Junyi Shen, Noppanat Wadlom, and Yao Lu. Batch query processing and optimization for agentic workflows. *arXiv preprint arXiv:2509.02121*, 2025.
- [14] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage*.
- [15] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137, 2024.

A 技术附录

表 4 汇总了下文使用的工作流和执行状态符号。表 5 汇总了扩展方法描述中使用的规划、评分和成本符号。

A.1 扩展问题建模

我们沿用正文的符号，考虑一个服务动态工作流实例集合的在线系统。令 $\mathcal{W}_t$ 表示时刻 t 的活跃工作流集合。为简化记号，阶段级和工作流级量均针对一个通用工作流实例定义，在含义明确时省略工作流索引。运行时就绪集合、前沿集合和可行动作集合，均理解为 $\mathcal{W}_t$ 中所有活跃实例对应集合的并集。

表 4: 符号汇总 I: 工作流和执行状态。

符号	含义
$\mathcal{W}_t$	时刻 t 的活跃工作流实例。
$G = (V, E)$	包含阶段 V 和先后依赖边 E 的工作流 DAG。
u, v	工作流阶段。
$(u, v) \in E$	从父阶段 u 指向子阶段 v 的依赖边。
$m(v)$	阶段 v 使用的模型类型。
$A(v) \subseteq \mathcal{D}$	阶段 v 的可用设备集合。
$R(v)$	阶段 v 的最大有界分片度。
$c_v(d)$	阶段 v 在设备 d 上的基础运行时间。
$\phi(v)$	阶段局部元数据和运行时注释。
$\text{Pa}(v), \text{Ch}(v)$	v 的父阶段集合和子阶段集合。
$s_t = (\rho_t, \kappa_t, \ell_t, \tau_t)$	时刻 t 的执行状态。
ρ_t	各设备的模型驻留状态。
κ_t	各设备的可复用前缀/缓存相关状态。
$\ell_t(u)$	保存父阶段 u 已完成输出的设备。
τ_t	执行状态中的设备可用性分量。
τ_d^{free}	设备 d 下一次可用的时刻。
$\mathcal{C}_t$	截至时刻 t 已完成的阶段。
$\mathcal{R}(s_t)$	状态 s_t 下的理想化就绪集合。
$\mathcal{F}(s_t)$	状态 s_t 下可行的阶段—设备候选集合。

每个工作流实例表示为有向无环图 (DAG) $G = (V, E)$, 其中每个节点 $v \in V$ 表示一个工作流阶段, 每条有向边 $(u, v) \in E$ 表示先后依赖: 只有阶段 u 的输出可用后, 阶段 v 才能开始。每个阶段 v 具有以下属性:

$$(m(v), A(v), R(v), c_v, \phi(v), \text{Pa}(v), \text{Ch}(v)).$$

这里, $m(v)$ 表示模型类型, $A(v) \subseteq \mathcal{D}$ 是可用设备集合, $R(v) \in \mathbb{Z}_{\geq 1}$ 是最大有界分片度, $c_v : A(v) \rightarrow \mathbb{R}_+$ 表示各可用设备上的基础运行时间画像, 满足

$$c_{\text{base}}(v, d) = c_v(d), \quad d \in A(v).$$

特征向量 $\phi(v)$ 包含阶段局部元数据, 例如提示词侧注释、共享前缀组信息、缓存相关标记及其他运行时注释。集合 $\text{Pa}(v)$ 和 $\text{Ch}(v)$ 分别表示 v 的父阶段和子阶段。

时刻 t 的执行状态记为

$$s_t = (\rho_t, \kappa_t, \ell_t, \tau_t),$$

其中, ρ_t 记录各设备的模型驻留情况, κ_t 记录各设备可复用的前缀相关元数据, ℓ_t 记录已完成父阶段输出所在的设备, τ_t 记录设备可用时刻。因此, s_t 概括了影响未来切换、传输、局部性和复用的系统状态。

令 $\mathcal{C}_t$ 表示截至时刻 t 已完成的阶段集合。在实现中, 运行时还跟踪正在执行的阶段, 以及已经由规划器提交但尚未完成的阶段。为使正文表述简洁, 正文省略了这些实现层面的集合。正文中的理想化就绪集合为

$$\mathcal{R}(s_t) = \{v \in V \setminus \mathcal{C}_t : \text{Pa}(v) \subseteq \mathcal{C}_t\},$$

而实现层面的就绪前沿还会排除已经运行或已经提交的阶段。

可行候选集合为

$$\mathcal{F}(s_t) = \{(v, d) : v \in \mathcal{R}(s_t), d \in A(v), (v, d) \text{ satisfies current resource feasibility}\}.$$

正文中的抽象状态相关成本函数 $g(\cdot)$ 可以具体化为以下可加的惩罚—收益分解:

$$\begin{aligned} \hat{c}(v, d, s_t) &= c_{\text{base}}(v, d) + c_{\text{switch}}(v, d, s_t) + c_{\text{transfer}}(v, d, s_t) \\ &\quad - b_{\text{locality}}(v, d, s_t) - b_{\text{prefix}}(v, d, s_t) - b_{\text{parallel}}(v, d, s_t). \end{aligned}$$

其关键性质在于: 调度决策会重塑未来执行状态, 进而改变下游的模型切换成本、传输成本、局部性和复用机会。

表 5: 符号汇总 II: 规划、评分和成本。

符号	含义
$\mathcal{R}_j$	规划步骤 j 的当前就绪前沿。
H	以 DAG 层级数表示的下游前瞻范围。
k	分片槽位索引, 其中 $k \in \{0, \dots, R(v) - 1\}$ 。
$y_{v,k,d}$	阶段 v 的分片槽位 k 在设备 d 上的二元分配变量。
$\Psi(v, k, d \mid s_t, H)$	状态 s_t 、前瞻范围 H 下的前沿规划得分。
$S(v, d \mid s_t)$	候选分配 (v, d) 的运行调度得分。
$\mathcal{A}_{\text{com}}$	规划器产生的已提交动作池。
$\mathcal{Q}(s_t)$	从 $\mathcal{A}_{\text{com}}$ 导出的依赖已就绪的排队动作集合。
$\hat{c}(v, d, s_t)$	将阶段 v 放置到设备 d 的状态修正成本估计。
C_{wait}	设备不可用造成的等待成本。
C_{switch}	模型切换成本。
C_{transfer}	从已完成父阶段跨设备传输的成本。
B_{colo}	与父阶段同置带来的收益。
B_{prefix}	前缀/缓存相关复用收益。
B_{parallel}	有界分片执行的收益。
$\beta_{i,j}$	从设备 i 到设备 j 的传输系数。
$\sigma(u, v)$	从 u 传给 v 的中间结果大小估计。
$\text{grp}(v)$	阶段 v 所属的共享前缀组。
$\text{Overlap}(\text{grp}(v), d, s_t)$	与设备 d 对齐的可复用共享前缀重叠量估计。
$\lambda_q, \lambda_s, \lambda_{\text{tr}}, \lambda_c, \lambda_p, \lambda_r$	等待、切换、传输、同置、前缀和并行各项的权重。
$\Delta_{\text{switch}}, \Delta_{\text{transfer}}, \Delta_{\text{locality}}, \Delta_{\text{prefix}}, \Delta_{\text{parallel}}$	成本估计器中依赖状态的修正项。

A.2 FATE 的扩展描述

FATE 包含三个相互作用的组件：利用前瞻候选评分的前沿 CP-SAT 规划器、提交并推进的调度构建过程，以及以状态为条件的成本和评分模型。规划器在当前就绪前沿上推理，选择阶段—设备分配，再将其具体化为各设备上可执行的队列。随后，当分配设备可用时，运行时执行器发出队列中依赖已就绪的阶段。

在每个规划步骤 j ，FATE 操作当前就绪前沿，而不是对完整工作流 DAG 求解一个整体优化问题。令 $\mathcal{R}_j$ 表示当前就绪前沿， H 表示下游层级前瞻范围，它仅用于在候选评分时估计未来尾部价值。也就是说， H 通过下游前瞻影响当前前沿分配的评价，但未来层级不会作为显式 CP-SAT 决策变量引入。

对于每个就绪阶段 $v \in \mathcal{R}_j$ 、每个分片槽位

$$k \in \{0, \dots, R(v) - 1\},$$

以及每个可用设备 $d \in A(v)$ ，我们引入一个二元决策变量

$$y_{v,k,d} \in \{0, 1\}.$$

变量 $y_{v,k,d} = 1$ 表示，在当前规划步骤，将阶段 v 的分片槽位 k 放置到设备 d 。

规划器求解以下前沿放置问题：

$$\max_y \sum_{v \in \mathcal{R}_j} \sum_{k=0}^{R(v)-1} \sum_{d \in A(v)} y_{v,k,d} \Psi(v, k, d \mid s_t, H),$$

其中， $\Psi(v, k, d \mid s_t, H)$ 是结合即时执行质量与下游尾部价值估计的状态感知放置得分。在实现中， Ψ 综合了基础执行质量，以及切换、传输、局部性、前缀相关复用、分片并行和模型专门化的影响。

决策变量受到设备容量、设备资格和有界分片约束。首先，在单次前沿规划轮次中，每个设备最多接收一个分片槽位分配：

$$\sum_{v \in \mathcal{R}_j} \sum_{k=0}^{R(v)-1} y_{v,k,d} \leq 1, \quad \forall d \in \mathcal{D}.$$

该约束仅适用于单次前沿规划轮次；在推进前沿或再次调用规划器后，可以进行更多分配。其次，一个阶段的每个分片槽位最多分配到一个可用设备：

$$\sum_{d \in A(v)} y_{v,k,d} \leq 1, \quad \forall v \in \mathcal{R}_j, k \in \{0, \dots, R(v) - 1\}.$$

第三，分片槽位按索引单调启用；只有选择了较低索引的槽位，才能选择较高索引的槽位：

$$\sum_{d \in A(v)} y_{v,k+1,d} \leq \sum_{d \in A(v)} y_{v,k,d}, \quad \forall v \in \mathcal{R}_j, k < R(v) - 1.$$

最后，只为 $d \in A(v)$ 定义变量，从而将分配限制到可用设备；等价地，也可以施加

$$y_{v,k,d} = 0, \quad \forall v \in \mathcal{R}_j, k \in \{0, \dots, R(v) - 1\}, d \notin A(v).$$

这些约束共同保证：当前规划轮次中每个设备最多接收一个阶段槽位分配，每个分片槽位最多分配到一个设备，主分配优先于额外分片槽位被选择，并且只有启用较低索引槽位后，才能启用较高索引槽位。

在当前前沿求解后，FATE 将选定的前沿分配具体化为各设备的可执行队列。当当前前沿处理完毕时，规划器推进 workflow 状态，重新计算下一个就绪前沿并重复上述过程。因此，规划采用滚动、逐阶段的方式：未来层级只通过前瞻评分影响当前选择，求解器的决策仍局限于当前前沿。

有界分片执行。 在我们的实现中，有界多设备执行通过子批次层面的查询分片实现。当某个就绪阶段选择了多个分片槽位时，该阶段的当前查询集合被划分为互不相交的查询分片，并分发到不同设备。每个设备在不同的查询子集上执行同一阶段。单个查询不会跨设备拆分，也不会为了竞速或推测完成而在多个设备上冗余执行。

相应的聚合也很简单：每个设备返回分配给它的分片中各查询的输出，然后按查询标识将输出合并回 workflow 状态。因此，有界分片执行改变的是当前批次在设备间的划分方式，不需要为单个查询进行跨设备归约。只有 $R(v) > 1$ 的阶段才能分片。在实践中， $R(v)$ 同时充当能力约束和策略上限：可划分阶段可以允许有界分片执行，而合并阶段或收尾阶段通常使用 $R(v) = 1$ 。

A.3 扩展调度得分定义

正文将调度得分定义为

$$\begin{aligned} S(v, d \mid s_t) = & -\lambda_q C_{\text{wait}}(v, d, s_t) - \lambda_s C_{\text{switch}}(v, d, s_t) - \lambda_{\text{tr}} C_{\text{transfer}}(v, d, s_t) \\ & + \lambda_c B_{\text{colo}}(v, d, s_t) + \lambda_p B_{\text{prefix}}(v, d, s_t) + \lambda_r B_{\text{parallel}}(v, d, s_t). \end{aligned}$$

下面更详细地定义各个分量。

令 t 表示当前调度决策时刻， τ_d^{free} 表示设备 d 下一次可用的时刻。等待成本为

$$C_{\text{wait}}(v, d, s_t) = \max\{0, \tau_d^{\text{free}} - t\}.$$

模型切换成本为

$$C_{\text{switch}}(v, d, s_t) = \begin{cases} 0, & \text{if } m(v) \text{ is already resident on } d, \\ \kappa_{\text{switch}}(m(v), d), & \text{otherwise.} \end{cases}$$

该项惩罚需要加载或激活当前未驻留在所选设备上的模型的放置。

跨设备传输成本估计为

$$C_{\text{transfer}}(v, d, s_t) = \sum_{u \in \text{Pa}(v)} \mathbf{1}[\ell_t(u) \neq d] \cdot \beta_{\ell_t(u), d} \cdot \sigma(u, v),$$

其中, $\ell_t(u)$ 表示父阶段 u 已完成输出所在的设备, $\sigma(u, v)$ 表示从父阶段 u 传给阶段 v 的中间结果大小估计, $\beta_{i,j}$ 是一对设备之间的传输系数。

为捕捉即时传输成本以外的结构局部性, 我们定义

$$B_{\text{colo}}(v, d, s_t) = \frac{1}{|\text{Pa}(v)|} \sum_{u \in \text{Pa}(v)} \mathbf{1}[\ell_t(u) = d],$$

约定当 $\text{Pa}(v) = \emptyset$ 时, $B_{\text{colo}}(v, d, s_t) = 0$ 。该项奖励使阶段靠近其已完成父阶段的放置, 从而保留与依赖关系一致的局部性。

前缀相关复用估计为

$$B_{\text{prefix}}(v, d, s_t) = \kappa_{\text{prefix}} \cdot \text{Overlap}(\text{grp}(v), d, s_t),$$

其中, $\text{grp}(v)$ 表示阶段 v 的共享前缀组, $\text{Overlap}(g(v), d, s_t)$ 估计已经与设备 d 对齐的可复用共享前缀结构量。该项是潜在复用的代理量, 而不是准确预测未来缓存命中的先知。

最后, 有界分片执行收益建模为

$$B_{\text{parallel}}(v, d, s_t) = \Delta_{\text{parallel}}(v, d, s_t),$$

其中, $\Delta_{\text{parallel}}(v, d, s_t)$ 估计在当前负载和依赖结构下, 可行的分片感知有界执行所带来的完成时间缩减。

A.4 扩展的状态条件成本模型

规划器和调度构建规则依赖状态修正成本估计。该估计器为规划器得分 Ψ 和调度得分 S 提供数值成本输入, 并非另一个独立的调度目标。其扩展形式为

$$\begin{aligned} \hat{c}(v, d, s_t) = & c_{\text{base}}(v, d) + \Delta_{\text{switch}}(v, d, s_t) + \Delta_{\text{transfer}}(v, d, s_t) \\ & - \Delta_{\text{prefix}}(v, d, s_t) - \Delta_{\text{locality}}(v, d, s_t) - \Delta_{\text{parallel}}(v, d, s_t). \end{aligned}$$

这些修正项捕捉以下因素: 所需模型是否已经驻留, 父阶段输出位于本地还是远端, 可复用共享前缀结构是否与候选设备对齐, 放置是否保留下游结构局部性, 以及有界分片执行是否缩短完成时间或改变下游竞争。

这种以状态为条件的视角是问题建模的核心: 两个其他方面完全相同的阶段—设备对, 可能因产生不同的未来执行状态而导致不同的下游成本。规划器利用这些修正估计, 在有限下游前瞻下比较前沿分配; 运行时执行器则遵循已经具体化的阶段—设备顺序, 同时满足依赖就绪和设备可用性要求。

A.5 完整过程描述

算法 2 比正文的紧凑版本给出了更详细的过程描述。选定放置首先具体化为已提交动作池 $\mathcal{A}_{\text{com}}$; 运行时从中导出依赖已就绪的排队动作集合 $\mathcal{Q}(s_t)$, 它不同于问题建模中的可行候选集合 $\mathcal{F}(s_t)$ 。一旦依赖和分配设备就绪, 运行时便发出排队阶段。

B 基线改编

为在共同执行底座上比较调度逻辑, 我们将所有基线实现于同一服务栈中。**Halo** 基线在该服务栈中忠实复现了原始 Halo 调度策略。对于完整系统无法直接兼容我们抽象的其他既有方法, 我们尽可能改编其策略, 而非复现完整系统。之所以需要这样做, 是因为据我们所知, 尚无现成系统能够联合提供我们所需的精确抽象: 异构多模型工作流 DAG、前缀/缓存状态代理、模型驻留、父阶段输出局部性、有界分片执行, 以及共同执行器。因此, 我们保留各改编方法的核心调度思想, 同时固定外围工作流运行时、基准模板、模型分配和执行环境。

算法 2 FATE: 完整调度过程

```

1: 初始化执行状态  $s_0$ 
2: 初始化已提交动作池  $\mathcal{A}_{\text{com}} \leftarrow \emptyset$ 
3: 当 活跃工作流实例中仍有未完成阶段 执行
4:   如果  $\mathcal{A}_{\text{com}}$  中没有可行的就绪动作 则
5:     构建当前就绪前沿  $\mathcal{R}_j$ 
6:     对于 每个  $v \in \mathcal{R}_j$ 、 $k \in \{0, \dots, R(v) - 1\}$ 、 $d \in A(v)$  执行
7:       计算状态修正成本  $\hat{c}(v, d, s_t)$ 
8:       计算前瞻感知候选得分  $\Psi(v, k, d \mid s_t, H)$ 
9:     结束 对于
10:    在  $(s_t, \mathcal{R}_j)$  上求解前沿 CP-SAT 规划器
11:    获得可用设备上的分片槽位放置
12:    将选定放置提交到  $\mathcal{A}_{\text{com}}$ 
13: 结束 如果
14: 从  $\mathcal{A}_{\text{com}}$  构建依赖已就绪的排队动作集合  $\mathcal{Q}(s_t)$ 
15: 根据具体化的各设备顺序和设备可用性, 发出就绪的排队动作
16: 在阶段完成时更新执行状态
17: 从  $\mathcal{A}_{\text{com}}$  移除已完成或不可行的动作
18: 结束 当
  
```

B.1 经典和实用基线

我们纳入以下经典和实用基线。**Halo** 忠实复现原始工作流服务启发式, 并直接实现在共同运行时中。**HEFT** 是针对本场景改编的经典异构 DAG 调度基线。**RR** 采用轮转方式进行放置和执行排序。

B.2 KVFlow 风格的未来复用感知缓存基线

KVFlow 风格基线是一种尽可能保留原有思想的改编: 它保留面向未来步骤的前缀优先级和缓存保留策略, 并结合贪心调度。具体来说, 它偏好能够为可能即将出现的步骤保留可复用前缀相关状态和同模型亲和性的动作, 但没有包含 FATE 使用的完整执行状态感知规划器。

B.3 Helix 风格的异构感知放置基线

Helix 风格基线是一种尽可能保留原有思想的改编: 它执行异构感知的最早完成放置, 并在分配时考虑传输和切换。它显式考虑设备异构性及即时分配成本, 但不建模当前决策之外的未来工作流执行状态。

B.4 原始方法与实现基线之间的映射

表 6 汇总了我们实现的基线与原始方法之间的关系。

表 6: 原始方法与共同服务栈中实现基线之间的映射。Halo 被忠实复现, 其他方法则针对共享抽象进行了改编。

原始方法	保留的核心思想	改编基线中未建模的部分
KVFlow	未来步骤感知的前缀优先级、缓存保留、同模型偏好	完整 KVFlow 缓存管理系统及端到端系统栈
Helix	异构感知放置、最早完成分配、传输/切换感知选择	完整异构服务系统及显式未来工作流状态建模
RoundRobin	在依赖就绪条件下简单轮转设备	状态感知放置、前缀复用和异构感知排序
HEFT	基于异构计算和传输代理的 DAG 排序与最早完成分配	LLM 特有的前缀复用及未来执行状态保留

表 7: 各实现方法获得的调度信号。所有方法使用相同的工作流执行器、生成模板、模型分配、查询工作负载和硬件环境。“驻留”“传输”和“前缀”表示调度期间相应代理信号是否可用。

方法	驻留	传输	前缀	前瞻机制
FATE	是	是	是	对运行时前沿进行显式未来状态规划。
Halo 风格	粗粒度	否	否	对 DAG 分配进行束搜索。
KVFlow 风格	是	部分	是	按缓存/复用优先级调度。
Helix 风格	是	是	否	异构感知放置。
HEFT	是	是	否	DAG 向上秩优先级。
RoundRobin	否	否	否	无前瞻。

B.5 基线的信号访问和公平比较范围

表 7 汇总了各实现策略可以使用的调度信号。我们的目标并非声称每个改编基线都完整重实现了相应生产系统。所有方法共享相同的执行器、工作流模板、模型分配、查询工作负载和硬件环境，仅在允许使用的调度信号和前瞻机制上有所不同。该设置在共同运行时中隔离出策略层面的行为。

主要的不对称在于：若干基线已经获得驻留或传输成本等即时状态信号，KVFlow 风格基线也被允许使用前缀复用信息。它们缺少的是 FATE 对前瞻感知前沿规划与运行时状态保留的联合使用，后者同时涉及模型驻留、父阶段局部性、前缀亲和性和有界分片机会。因此，我们将性能提升理解为联合未来状态感知调度的证据，而不是每个单独代理信号都属于 FATE 独有的证据。

C 补充实验细节

C.1 基准构建细节

我们构建两个基准套件：从 WfCommons/WfInstances 派生的工作流 DAG 套件，以及受控前缀复用套件。前者使用 WfCommons/WfInstances 的 JSON 工作流描述来获得现实依赖结构，后者固定工作流模板，并改变重复长上下文前缀结构的数量。两个套件均由固定模板、稳定哈希和固定构建种子确定性生成。所有调度方法使用相同的生成工作流实例、模型分配、运行时代理量和缓存相关元数据。

WfCommons/WfInstances 的选择和筛选。 对于基于 WfCommons/WfInstances 的基准，我们解析配置的原始数据根目录下的工作流实例 JSON 文件，根据任务依赖字段构建有向无环图。解析器兼容各 WfCommons 版本使用的 task、job 和 node 格式，提取任务标识与父依赖，并依据工作流族、评估分组、折叠后图大小、图深度、图宽度、各族数量上限及实例总数上限进行显式筛选。固定的论文实验清单记录了报告结果所使用的实例文件和筛选设置。

我们将 WfCommons/WfInstances 用作现实工作流依赖结构的来源，而非 LLM 执行的直接轨迹。原始工作流任务被映射为 LLM 阶段组，并附加模型、运行时、通信和缓存相关元数据。这使基准适合受控比较调度策略，同时避免声称每个原始 WfCommons 任务都作为一次独立 LLM 调用被重放。

DAG 转换和阶段组构建。 首先将每个原始工作流实例解析为任务 DAG。为获得 LLM 阶段执行图，基准构建器将原始任务映射为阶段组。当原始任务名称族重复出现时，构建器确定性地具有相同规范化任务名称前缀的任务折叠为阶段组。转换后的 DAG 保留组间诱导依赖：只要 u 中任一原始任务是 v 中任一原始任务的父任务，就添加从组 u 到组 v 的边。为避免超大工作流出现退化的过度压缩，当直接按前缀折叠会产生过少组时，实现会对分组阶段的数量设置上限。

随后，依据拓扑结构为每个转换后的阶段标注入度、出度、层级、反向深度和层宽。这些结构注释仅用于分配工作流角色和代理成本。所得图保留转换后阶段组之间的依赖关系，但不是逐一重放每个原始任务。

阶段角色模板。 基准通过确定性的结构规则，为每个转换后的阶段分配角色模板。构建器使用拓扑层级、入度、出度、汇入/分出模式及规范化任务名称族等信息。类似源节点的阶段和早期宽层阶段映射为提示词准备、检索、路由或分解角色；具有明显分出的中间阶段映射为工作者角色；高汇入阶段映射为合并或聚合角色；类似汇节点的阶段或较晚阶段映射为摘要、验证、核验或最终综合角色。

角色分配不针对任何调度器优化。它是用于附加 LLM 服务属性的固定模板，这些属性包括预期预填充/解码工作量、输出大小代理量、通信权重、缓存行为、候选模型，以及是否允许有界数据并行。所有调度器都操作相同的角色标注工作流实例。

模型分配。 每个转换后的阶段根据其角色分配一个模型别名。各角色的候选集合在工作流中引入模型异构性。模型别名包括 Qwen 风格、DeepSeek 风格和 Llama 风格的 7-8B 画像。部分面向缓存、长上下文或特定工作流族的评估分组使用固定模型别名，以隔离局部性和前缀复用行为；通用阶段则通过确定性的稳定哈希，在依赖角色的候选模型中选择。WfCommons 转换脚本在此分配中使用固定种子 20260423。

这种确定性分配确保所有调度方法看到相同的模型分布，并使基准生成可复现。模型分配不会针对某个调度器单独优化，也不允许调度器选择阶段所分配的模型。

运行时间、切换、传输和前缀/缓存代理量。 由于源工作流轨迹并非 LLM 服务轨迹，执行成本通过受控代理量表示。每个模型画像规定模型规模、内存占用、预填充/解码系数和模型切换惩罚。每个阶段角色提供复杂度乘数、预填充/解码缩放因子、最大 token 数代理量、输出大小代理量和通信权重。

传输压力通过父子依赖和输出大小代理量建模。若子阶段被放置在与父阶段不同的设备上，其估计传输成本高于同置子阶段。传输成本由父阶段输出 token 代理量和常数边传输系数估计，因此它是相对局部性代理，而非实测互连带宽模型。

模型驻留连续性通过切换成本代理量和同模型续接特征建模。前缀/缓存行为通过 `shared_prefix_group`、`keep_cache` 和 `cache_reuse` 等阶段字段表示。所得缓存得分估计后续阶段能否在同一设备复用兼容前缀状态；它不是硬件 KV 缓存命中计数器。

对于每个阶段 v ，当阶段级并行执行适用时，基准还规定有界分片度 $R(v)$ 。合格阶段最多可使用 $R(v)$ 个复制执行槽位；在报告的实验中， $R(v) \leq 2$ 。它建模有限的阶段级数据并行，而不是将单个模型进行张量并行划分。

受控前缀复用工作负载。 除 WfCommons 派生基准外，我们还根据工作流风格 DAG 模板和 LongBench 风格 [15] 文本池构建受控前缀复用套件。生成器以 0、0.25、0.5 和 1.0 为目标比例，使用固定前缀长度和确定性分组创建重复共享前缀组。重复比例越高，越多查询被分到共享前缀组，从而为缓存或前缀感知策略保留有用状态提供更多机会。

该套件用于行为分析，而非作为额外的真实轨迹基准。它用于隔离 FATE 的优势是否只来自前缀复用，还是来自联合平衡前缀亲和性、父子局部性、模型驻留连续性、传输压力和有界分片执行。因此，这些工作负载应理解为受控前缀复用压力测试，而非额外的 WfCommons 真实 DAG 轨迹。

基准局限性。 该基准保留工作流 DAG 结构，但不声称重现生产 LLM 轨迹。运行时间、传输、缓存和模型切换成本都是从阶段角色和模型画像派生的代理特征。基准也不建模突发生在线到达、内存碎片、依赖 tokenizer 的 KV 缓存布局或实测设备间带宽。因此，其用途是在固定工作流和状态代理下受控比较策略，而不是端到端预测某个具体集群的服务延迟。

C.2 指标与聚合

令 $T_{m,i}$ 表示方法 m 在工作流实例 i 上的工作流总完成时间， $T_{RR,i}$ 表示对应的 RoundRobin 总完成时间。我们以几何平均报告归一化总完成时间：

$$\text{NormMS}(m) = \exp \left(\frac{1}{N} \sum_{i=1}^N \log \frac{T_{m,i}}{T_{RR,i}} \right).$$

对于查询级尾部性能，令 $L_{m,i}^{95}$ 表示方法 m 在实例 i 上的查询完成延迟第 95 百分位。归一化 P95 延迟为

$$\text{NormP95}(m) = \exp \left(\frac{1}{N} \sum_{i=1}^N \log \frac{L_{m,i}^{95}}{L_{\text{RR},i}^{95}} \right).$$

为诊断机制，我们报告三个按任务归一化的代理指标。跨设备边率为

$$\text{XDevEdge}(m) = \frac{\sum_i \text{cross_device_parent_edges}_{m,i}}{\sum_i \text{workflow_tasks}_i}.$$

缓存得分为

$$\text{CacheScore}(m) = \frac{\sum_i \text{prefix_cache_hits_est}_{m,i}}{\sum_i \text{workflow_tasks}_i}.$$

同模型续接指标为

$$\text{ModelCont}(m) = \frac{\sum_i \text{same_model_continuations}_{m,i}}{\sum_i \text{workflow_tasks}_i}.$$

这些机制指标仅用于解释调度器行为。较低 XDevEdge 表示跨设备父子放置较少；较高 CacheScore 表示估计的前缀/缓存亲和性更强；较高 ModelCont 表示模型驻留连续性更强。

C.3 消融定义

消融研究每次禁用 FATE 的一个组件，同时固定基准、设备设置、模型分配和其余调度器参数。

无未来规划变体禁用 FATE 使用的未来状态规划路径：有效规划范围缩减为一，因此求解器仍分配当前就绪前沿，但不再使用即时前沿之外的下游前瞻。它应理解为移除未来状态规划，而不是缩短 CP-SAT 前瞻范围。无局部性项变体禁用父阶段局部性、传输和远端父阶段放置项。无同模型奖励变体移除继续使用所选设备已经偏好或已经驻留模型的奖励，但保留模型切换成本。无前缀/缓存项变体禁用前缀/缓存感知评分。无分片并行变体禁用有界多设备分片执行。

这些消融用于检验 FATE 的提升来自某一个主导信号，还是来自联合保留执行状态的多个维度。

C.4 评估流程和结果来源

报告结果围绕一条可审计流程组织。首先，基准构建脚本生成固定 workflow 模板和固定实验清单。其次，清单运行器在相同运行时框架下执行每个调度器，为每个实验导出一个 CSV 文件。最后，单一论文表格导出器仅消费这些清单声明的 CSV 文件，并按照附录 C.2 的指标定义，计算主表、ECDF 输入、消融表、前缀复用表和机制指标汇总。无法从固定清单和导出器追溯的结果不会用于论文表格。

这条流程旨在区分原始运行日志与论文证据。实验清单定义 workflow 实例集合、调度器变体、查询数量、设备配置及输出 CSV 路径；导出器定义归一化基准、几何平均聚合及机制代理公式。这使每个报告数值的来源明确：每个表项都能追溯到一个方法、workflow 实例、原始 CSV 行和聚合公式。

C.5 可复现细节

基准构建采用确定性规则和固定构建种子。所有比较方法使用相同的生成 workflow 模板和查询工作负载。主要实验运行在一台服务器上，配置为八张 NVIDIA RTX 4090 GPU、AMD EPYC 7763 64-Core Processor，以及 503 GB 系统内存。单次基准运行通常需要约 2–5 分钟，具体取决于 workflow 套件、批大小和调度器配置。在该服务器上，完整报告实验约需 2–3 天。发布工件的 README 给出准确的环境配置、基准生成命令、清单路径、调度器参数及导出器命令，可据此从原始运行日志复现论文表格和图。

D 补充结果

D.1 按 workflow 族分解

表 8 按 workflow 族报告 workflow DAG 基准中的归一化总完成时间。数值为各族内部的几何平均，且仅使用所有比较方法均已完成运行的严格交集。该分析应针对论文使用的转换后 LLM 阶段 workflow 模板理解，而非针对原始 WfCommons 任务。

按族分解揭示了三类主要结构情形。第一，SeismicCrossCorrelation 和 Srasearch 等缓存主导 workflow 族，转换后的 DAG 较浅或较小，模型多样性低，同模型连续性强。在这些族中，复用感知或局部性感知基线已经捕捉到部分可利用结构；但更新结果表明，缓存主导结构并未消除未来状态感知规划的价值：FATE 仍取得明显提升，尤其是在 Srasearch 上，这说明即便前缀复用丰富，保留下游放置和模型状态连续性仍可能重要。第二，BWA 和 BLAST 等非常宽的分支—合并族暴露出的未来状态连续性较弱：许多就绪阶段可以并行调度，承载缓存的结构有限，主要挑战往往是即时负载均衡或最早完成放置。在这种情形下，FATE 并非始终占优，较强的非 FATE 基线能够与之持平或略胜。第三，Montage、SoyKB 和 Nextflow 等更深的多模型族会形成更长的下游影响链，当前放置会在多个后续阶段影响模型驻留、传输和合并局部性。这些情形仍有利于 FATE，较大提升出现在深层多模型族和结构化缓存主导族中。

其余族，包括 1000Genome 和 Cycles，位于上述极端之间。它们具有足够的工作流结构，使未来状态感知规划有帮助，但不足以产生 Montage、SoyKB 或 Srasearch 那样大的优势。因此，按族结果支持一个依赖工作负载的结论：当 workflow 结构产生不平凡的多步局部性、驻留或复用权衡时，未来状态保留最有价值；当转换后的 DAG 高度对称且宽度占主导时，其决定性较弱。我们还提醒，RNA-seq 子集较小，因此其族级结果应被视为描述性证据，而非具有统计决定性的结论。

我们也指出，为控制评估预算，固定论文清单将转换后 workflow 大小限制为最多 64 个阶段。因此，本附录描述的是报告的基准子集，并非所有可能的工作流规模。更大的转换后 DAG 可能暴露更长的下游状态依赖，并改变未来状态感知规划的相对价值；但这仍是未来评估问题，不能作为当前研究的直接结论。

表 8: workflow DAG 基准中按族分解的归一化总完成时间。数值为族内几何平均，仅使用所有比较方法均完成运行的严格交集。越低越好。“最佳非 FATE 对照”表示同一族中的最强基线。

workflow 族	DAG 数	FATE	最佳非 FATE 对照
1000Genome	22	0.742	0.800 (Helix)
BLAST	15	0.712	0.707 (HEFT)
BWA	15	0.810	0.768 (KVFlow)
Cycles	19	0.778	0.792 (KVFlow)
Montage	12	0.535	0.600 (HEFT)
Nextflow	9	0.708	0.725 (Helix)
RNA-seq	3	0.688	0.738 (HEFT)
SeismicCrossCorrelation	11	0.762	0.847 (KVFlow)
SoyKB	10	0.550	0.739 (HEFT)
Srasearch	25	0.531	0.643 (Helix)

D.2 受控冲突压力测试

作为仅在附录报告的诊断扩展，我们构建受控冲突压力测试，以隔离主受控前缀复用套件未重点考察的情形。主前缀复用套件有意对缓存友好：同模型局部性、父阶段局部性和重复前缀结构大体一致。相较之下，受控冲突套件有意使这些信号相互矛盾。

具体而言，workflow 是一条固定的模型专门化阶段链，在多个模型族之间交替，同时始终保留缓存相关状态。因此，调度器反复面临多步权衡：将当前阶段放置在即时父阶段附近有利于短期局部性，但可能破坏某个模型的驻留状态；当 workflow 数步后返回较早的模型族时，该驻留状态本来会更有价值。这正是短视局部性启发式难以处理、而有限前瞻未来状态感知调度器试图解决的状态权衡。

为使工件清晰，该套件由主受控前缀复用研究使用的同一 workflow 基准生成器产生，但走一条独立的受控冲突模板路径。工件生成四个仅用于附

录的模板：workflow_cache_conflict_000.yaml、workflow_cache_conflict_025.yaml、workflow_cache_conflict_050.yaml 和 workflow_cache_conflict_100.yaml，分别对应共享前缀重复比例 0、0.25、0.5 和 1.0。这些运行通过独立的附录实验清单评估，不纳入正文汇总基准表。

表 9 表明，这种有意构造的对抗情形产生了比主受控前缀复用套件大得多的差距。我们强调，这是预期结果，也正是该套件有用的原因：它放大驻留与局部性的冲突，而不让这种冲突在更温和工作流的平均中消失。因此，所得差距应理解为机制层面的支持证据，而非旨在反映自然工作负载频率的核心基准结果。

随重复比例变化的趋势也提供了信息。FATE 在重复比例 0 时已经占优，此时不存在额外共享前缀复用，说明该套件中的主要作用不只是缓存复用。将重复比例从 0 增至 1.0，仅使 FATE 进一步获得小幅改善，而 Halo 和 KVFlow 风格基线相对平稳。这说明，额外前缀复用本身并不能解决所诱导的即时父阶段局部性与未来模型专属执行状态保留之间的冲突。主要收益来自能够在未来多个步骤保留有用下游驻留和放置结构的规划决策。

由于该构造刻意具有对抗性，我们不将表中绝对差距解释为主基准的代表。我们用它支持一个更窄的结论：当有意使前缀局部性、父阶段局部性和未来模型驻留发生冲突时，未来状态感知规划可能远比单独的复用感知调度重要。

表 9: 仅在附录报告的受控冲突压力测试。工作流沿链条交替执行模型专门化阶段，有意制造即时父阶段局部性与未来模型驻留之间的矛盾。数值为本套件内部相对于 Halo 的归一化总完成时间。越低越好。

方法	0	0.25	0.5	1.0
Halo	1.00	1.00	0.99	0.99
KVFlow	1.00	0.98	0.98	0.97
FATE	0.30	0.27	0.27	0.26

D.3 方差与稳定性

基准构建、实验清单、模型分配和查询工作负载均固定。我们在相同固定清单下将所有报告实验重复两次。剩余的运行间变化主要来自服务系统初始化、模型加载、GPU 运行时影响和进程调度。方差分析使用两次重复都已完成的工作流实例和调度器变体的严格交集。

在重复实验中，原始墙钟总完成时间的变异系数中位数为 1.40%，均值为 1.43%，最大值为 3.82%；两次重复的相对差幅中位数为 2.80%，最大值为 7.65%。将每次重复运行用相同工作流、相同重复编号下的 RoundRobin 运行归一化后，配对变异系数中位数为 1.31%，均值为 1.49%，最大值为 4.68%。因此，我们认为较大的汇总差距体现有意义的系统级效应，而将接近该噪声水平的小差异报告为描述性结果，而非具有统计决定性的证据。

D.4 超参数敏感性

我们进行了小规模超参数敏感性检查，检验 FATE 的提升是否依赖某个脆弱的单一前瞻范围或评分权重。实验使用 30 个按族均衡的 WfCommons 派生 DAG。该子集从工作流 DAG 清单中确定性选取大小接近目标 24 阶段的实例。该检查用于稳健性分析，而不是额外调参。

表 10 报告同一工作流中相对于 RoundRobin 的归一化总完成时间。主实验默认设置为 $H = 4$ 。 $H = 0$ 行禁用未来状态规划， $H = 1$ 、 $H = 2$ 和 $H = 3$ 则缩小前瞻范围。其余行扰动运行时评分项的不同分组：状态项缩放同模型项和模型切换项，局部性项缩放父阶段同置及传输相关项，前缀项缩放前缀/缓存复用项。

在该子集上，所有测试配置仍显著低于 RoundRobin。默认 $H = 4$ 并非这些 DAG 上唯一最优的设置，这符合预期，因为该子集并不用来调参，而且短前瞻可能在较小或较浅的图上具有竞争力。关键观察是：在前瞻范围和权重扰动下，性能仍保持在较窄区间。因此，我们将该实验理解为 FATE 的行为并非由单一神奇权重解释的证据；规模更大的消融套件仍是各机制贡献的主要证据。

表 10: 30 个 WfCommons 派生 DAG 上的超参数敏感性。数值是用同一工作流的 RoundRobin 总完成时间归一化后的几何平均。最后一列将各设置除以默认 $H = 4$ 设置。越低越好。

设置	归一化总完成时间 ↓	相对于 $H = 4$ ↓
$H = 0$, 无未来状态规划	0.760	1.063
$H = 1$	0.706	0.987
$H = 2$	0.710	0.993
$H = 3$	0.726	1.015
$H = 4$, 默认	0.715	1.000
状态项 $\times 0.5$	0.697	0.975
状态项 $\times 1.5$	0.731	1.022
局部性项 $\times 0.5$	0.723	1.011
局部性项 $\times 1.5$	0.718	1.004
前缀项 $\times 0.5$	0.721	1.008
前缀项 $\times 1.5$	0.731	1.022

表 11: 基准中按族分层的 30-DAG 子集上的代理成本扰动。数值是在相同扰动条件下用 RoundRobin 总完成时间归一化后的几何平均。越低越好。

条件	FATE ↓	KVFlow ↓	Helix ↓
默认	0.691	0.749	0.773
切换成本 $\times 0.5$	0.711	0.743	0.807
切换成本 $\times 2.0$	0.718	0.755	0.800
传输成本 $\times 0.5$	0.715	0.755	0.804
传输成本 $\times 2.0$	0.710	0.737	0.779
前缀收益 $\times 0.5$	0.699	0.733	0.760
前缀收益 $\times 2.0$	0.695	0.739	0.775

D.5 代理成本扰动

我们进一步检验 FATE 的优势是否依赖特定的运行时代理画像校准。基准模板使用模型切换、父阶段传输/通信压力及前缀/缓存复用收益的代理成本。为对这些假设施加压力，我们从工作流 DAG 基准构建一个包含 30 个 DAG、按族分层的子集，分别将每类代理乘以 0.5 和 2.0，同时固定工作流结构、查询数量和调度器配置。

表 11 报告在相同扰动条件下相对于 RoundRobin 归一化的总完成时间几何平均。FATE 在所有扰动下仍是最强方法。绝对归一化总完成时间的变化较小，从默认代理画像下的 0.691，最多增至测试扰动下的 0.718。这表明，观察到的优势并非某一个切换成本、传输成本或前缀收益校准的产物。相反，无论代理成本被低估还是高估，FATE 联合处理未来模型驻留、父阶段局部性、传输压力和前缀/缓存亲和性仍有益。

前缀各行应理解为对模板级前缀/缓存收益代理量的扰动，不能替代表 2 的受控前缀复用研究。受控前缀复用套件改变可复用前缀结构的数量，而本实验改变的是现有工作流 DAG 模板所附成本的校准。

D.6 求解器开销分解

在用于诊断、已完成的求解器支持工作流 DAG 和消融 CSV 文件中，FATE 共记录 2175 次 CP-SAT 前沿求解。所有记录求解均返回 OPTIMAL。求解器墙钟时间均值为 0.0059s，中位数为 0.0042s，P95 为 0.0142s，最大值为 0.0754s，记录的目标差距均为零。开销与工作流运行时间分开报告，以免混淆调度成本和执行延迟。

表 12: 已完成的求解器支持诊断运行中的 CP-SAT 前沿规划开销。

求解次数	最优次数	均值 (秒)	中位数 (秒)	P95 (秒)	最大值 (秒)
2175	2175	0.0059	0.0042	0.0142	0.0754

D.7 更广泛的影响

本工作研究异构 LLM workflows 服务系统的调度。潜在积极影响包括提高服务效率、减少资源浪费，以及降低 workflow 式 LLM 应用的成本和能耗。这些改善可能使计算预算有限的研究者和较小组织更容易使用复杂 LLM workflow。

本工作没有引入新的模型能力、数据采集机制或面向用户的应用。但效率提高也可能增加 LLM 服务总需求，部分抵消每次查询的节能收益。这些风险是间接的，并取决于下游应用。适当的缓解措施包括：将调度器用于已为底层 LLM 应用实施现有安全策略、访问控制、监控、速率限制和隐私保护的系统。

译者补充：原图英文标注对照

图 1 (系统概览)。 Input: Heterogeneous LLM Workflow DAG: 输入: 异构 LLM workflow DAG; Model Types: 模型类型; Input Summarization: 输入摘要; Translation: 翻译; Knowledge Extraction: 知识提取; Chat: 对话; Response Planning: 响应规划; Output Formatting: 输出格式化。模型名 Llama、Qwen、DeepSeek 原样保留。

Execution State: 执行状态; Model Residency: 模型驻留; Parent-output Locations: 父阶段输出位置; Shared-prefix overlap: 共享前缀重叠; Downstream Reachability: 下游可达性。

FATE: Future-State-Aware Scheduler: FATE: 未来状态感知调度器; Horizon-Aware Planner: 前瞻范围感知规划器; Optimizing over downstream DAG levels: 针对下游 DAG 层级优化; Inputs: 输入; Summarization: 摘要; Upcoming Tasks: 即将到来的任务; Time: 时间; Current Execution State: 当前执行状态; Committed Frontier Assignments: 已提交的前沿分配。

Assignment Scoring: 分配评分; Immediate cost and future-state value: 即时成本与未来状态价值; GPU0: Higher Score: GPU0: 得分较高; GPU1: Lower Score: GPU1: 得分较低; Compute Efficiency: 计算效率; Data Locality: 数据局部性; Prefix Reuse: 前缀复用; Transition Overhead: 转换开销。

Multi-Device Shard Execution: 多设备分片执行; Assigned Task: 已分配任务; Runtime Feedback: 运行时反馈; Device Load: 设备负载; Memory Usage: 内存使用量; Progress: 进度; Runtime State Update: 运行时状态更新; Cache: 缓存; Ready Tasks: 就绪任务。

Myopic Queue Scheduling: 短视队列调度; Current Ready Tasks: 当前就绪任务; Task A/B/C: 任务 A/B/C; Myopic Scheduler: 短视调度器; Poor locality: 局部性差; Model switch: 模型切换; Low reuse: 复用少; FATE Scheduling: FATE 调度; Fewer switches: 更少切换; Less movement: 更少数据移动; End-to-End Workflow Latency: 端到端 workflow 延迟; Myopic: fragmented, switch-heavy: 短视策略: 执行碎片化、切换频繁; FATE: clustered, locality-aware: FATE: 聚集执行、感知局部性。时间点及 GPU 编号保持原样。

图 2 (经验累积分布)。 Per-workflow normalized makespan (vs. RoundRobin): 每个 workflow 相对于 RoundRobin 的归一化总完成时间; Cumulative fraction of workflows: workflow 累积比例; RoundRobin baseline: RoundRobin 基准线。FATE、Helix、KVFlow、Halo 名称及全部坐标数值原样保留。

译者补充：公式文字与阅读说明

公式中的英文条件保持原式，中文对应如下：satisfies current resource feasibility 表示“满足当前资源可行性”；if ... is already resident on ... 表示“如果模型已经驻留在该设备上”；otherwise 表示“否则”。指标中的 cross_device_parent_edges、workflow_tasks、prefix_cache_hits_est、same_model_continuations 分别表示跨设备父依赖边数、 workflow 任务数、估计前缀缓存命中数和同模型续接次数。

1. 前缀收益公式使用 $grp(v)$ ，紧接的解释使用 $g(v)$ 。译文按原文保留这一符号差异，未额外推断两者的定义。
2. 消融定义把“无未来规划”的有效范围描述为一；超参数敏感性表则用 $H = 0$ 标记禁用未来状态规划，并另外报告 $H = 1$ 。译文保留两处原始表述，复现实验时需核对工件中禁用路径与范围参数的对应关系。
3. XDevEdge 的分母是任务数，不是依赖边总数，因此该指标可以大于一。CacheScore 是估计复用代理量，并非硬件 KV 缓存实际命中率。
4. CP-SAT 返回 OPTIMAL 指当前前沿优化模型的最优状态，不代表完整 workflow 的全局最优调度。未来层级通过候选评分进入，未成为显式未来决策变量。
5. 评估中的依赖结构来自 WfCommons/WfInstances，但角色、模型和部分成本通过确定性模板与代理画像附加。KVFlow、Helix、HEFT 的实现范围及真实轨迹、突发到达、成本代理的局限均按原文保留。附录说明求解器开销与 workflow 运行时间分开报告。